

Design Pattern

...

2025 J4

Design pattern powa

Design Pattern

Design Pattern

Definition ?

Uses et coutumes des développeurs, une pratique tellement répandu qu'elle a fini par être conceptualisé pour être standardisé pour répondre à des besoins spécifiques.

Il faut faire cependant à ne pas tomber dans la sur-analyse, de la sur-conceptualisation. Les solutions envisagées doivent être proportionnées au problème et aux problématiques d'architecture et de maintenabilité.

Les 23 Design Patterns du GoF

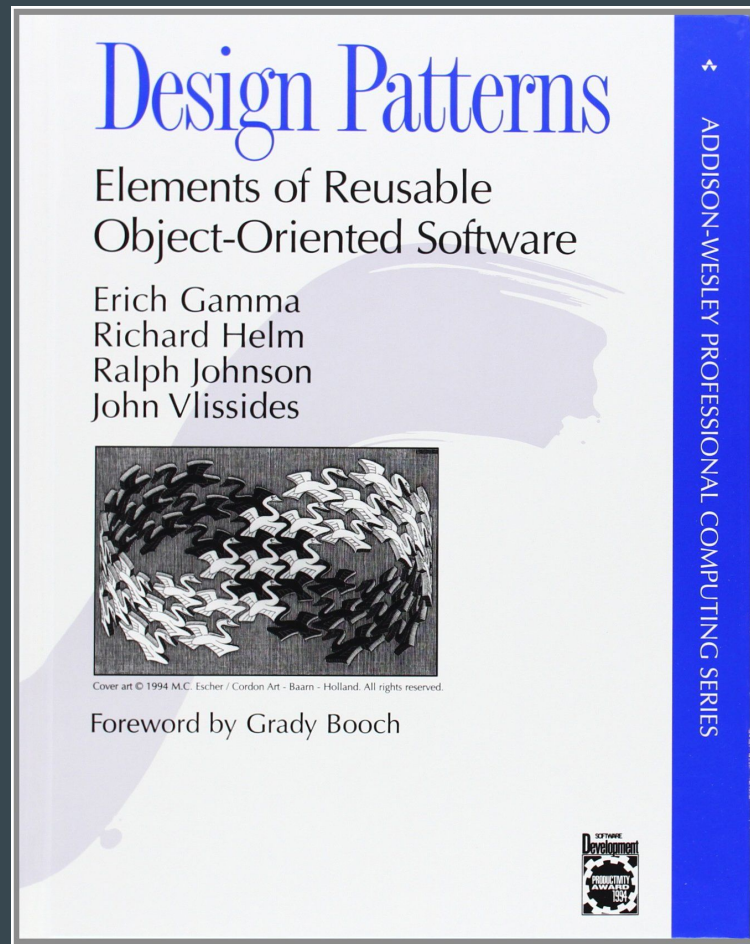
1994

Erich Gamma

Richard Helm

Ralph Johnson

John Vlissides



L'exemple typique du Design Pattern

Singleton.

Problématique : J'aimerais avoir un accès rapide, simple à un composant donné.

Ce composant n'existe qu'en un seul exemplaire au runtime. Promis.

Implémentation classique

```
class Singleton<T>
{
    public static T Instance { get; private set; }

    public static void SetInstance(T instance)
    {
        Instance = instance;
    }
}
```

Initialisation

```
public class Player : MonoBehaviour
{
    void Start()
    {
        Singleton<Player>.SetInstance(this);
    }
}
```

Utilisation

```
Singleton<Player>.Instance.Blablabla();
```


Facile...

- Le singleton, initialement partagé dans le livre des design patterns, est depuis décrié. Les auteurs se sont excusés.
 - C'est un anti-pattern
-
- Le but était d'avoir un code clean, qui a juste les accès dont il a besoin et qui a des dépendances explicites...
 - Le Singleton peut être utilisé par n'importe qui, n'importe quand, pour n'importe quoi. Surtout n'importe quoi
 - Loi de Murphy, si un truc peut mal se passer ... ça se passera.

Loi de Murphy

“S'il existe au moins deux façons de faire quelque chose et qu'au moins l'une de ces façons peut entraîner une catastrophe, il se trouvera forcément quelqu'un quelque part pour emprunter cette voie.”

S Pen du Galaxy Note 5 Samsung ...



Warning: Be sure to insert your S Pen with the nib pointed inward. Inserting the S Pen the wrong way can cause it to become stuck and can damage the pen and your phone.

Un peu de tempérance

Mais le singleton c'est pratique, ya pas à réfléchir à comment fournir l'instance, c'est rapide ... ça peut être utile, mais c'est à utiliser avec parcimonie.

Game Loop, le concept

Le programme doit être constamment actif, donc on tourne au sein d'une boucle infinie.

Dans cette boucle on répète tout le temps les mêmes opérations :

- On récupère les input
- On met à jour nos entités
- On met à jour l'écran de l'utilisateur

On répète cette séquence tant que l'on a pas d'ordre d'extinction devant de l'OS ou de l'utilisateur.

Sur Unity, ils vont un peu plus loin

Aussi vite que possible

- Input
- Update
- Coroutine
- LateUpdate
- Draw
- Wait si VSync

À interval le plus régulier possible

- FixedUpdate
- UpdatePhysic
- OnTrigger/Collision

C'est de là que viens le deltaTime

- `Time.deltaTime` indique le temps total de la précédente update

Admettons un objet se déplace à 1 unité par seconde, s'il s'est passé 1 seconde ou 2 secondes le résultat n'est pas le même.

ça nous permet de jouer avec un framerate inconstant qui va toujours au “plus vite”.

Même histoire pour fixedDeltaTime

- Il est plus efficient de faire tourner la physique sur un autre thread mais ça nous oblige de décorréler l'update du calcul de la physique. Plus le calcul de la physique est régulier mieux c'est.
- Malgré cette régularité souhaitée, on peut toujours avoir un léger décalage
fixedDeltaTime est donc toujours d'actualité
- Dans le temps qui sépare deux fixed update, il peut y avoir
 - 0 update (retard de rendu ou script trop long)
 - 1 update (le meilleur des cas)
 - voire plusieurs updates (possibilité de clipper, de zapper des event physique et d'avoir un jeu moins "smooth" car la physique a un retard)

Concrètement, que faire dans l'update, que faire dans fixedUpdate ?

On veut gérer les inputs le plus rapidement possible

- Gestion des inputs dans l'update
- La mise à jour générale des entité dans l'update (sauf déplacement si on veut utiliser la physique)

On veut une cohérence physique la plus clean possible

- Les déplacements se gèrent souvent dans la phase physique
- Mais en cas de framerate élevé (donc plusieurs update pour 1 fixedUpdate) on a un effet de déplacement saccadé.
 - Solution : prévoir la prochaine position de la physique et ajouter un mécanisme d'interpolation dans l'update qui va fluidifier le mouvement
 - Ou déplacer le calcul de la physique dans l'update dans les project settings

Ce n'est pas grave de mettre l'ensemble des mises à jour dans l'Update

- Le fonctionnement avec fixed Update nous permet d'être le plus précis (cas des jeux qui se veulent exigeant comme les jeux de combat).
- ... Ou alors maintenant on peut faire calculer la physique lors de l'Update
- Par contre, le comportement de l'update qui va le plus vite possible, ça pose souvent soucis...
 - Aller le plus vite veut dire que l'on monte la charge au maximum sur notre machine, notre CPU ou notre GPU fini par être à 100%, pas top pour la consommation et le "max de frame" n'est pas toujours un besoin (ex: jeux contemplatif, tour par tour, top-down, jeux mobile).
 - `Application.targetFrameRate`

La GameLoop à notre niveau

- Dans nos niveaux, on aime souvent avoir un composant “chef d’orchestre”.
 - On peut se baser sur le modèle de la GameLoop pour en implémenter une à notre niveau afin de bien synchroniser nos entités.
 - Coroutine, connaît l’ensemble des entités en jeu. Met à jour l’ensemble périodiquement.
 - Permet d’avoir le contrôle sur la mise à jour des entités, si on est en pause : plus de mise à jour.
 - Permet d’affiner la mise à jour des entités. Est-ce nécessaire de mettre l’entité à 100 unité du joueur ?

ServiceLocator

Nous permet de récupérer une instance donnée d'un type.

Web : Donne moi le Logger, donne moi un accès à un générateur pseudo-aléatoire etc

JV : Donne moi le component Player associé à ce GameObject, donne moi le composant

Principale différence avec le Singleton

- N'est pas statique.

GetComponent, GetComponentInChildren, GetComponentInParent

Mais sur ce coup, Unity ...

Dépendant de l'organisation / l'arborescence de la scène

- Le coup classique, quelqu'un bouge un composant dans un autre gameObject, et c'était pas prévu par un script qui en dépendait.

-> Solution à ça, éviter le GetComponent quand on peut, exemple, préférer l'utilisation de champs. Dans le cas d'un Trigger/Collider, on va forcément faire un GetComponent.

Une autre manière de faire, utiliser un ScriptableObject = SOAP

Seul ceux qui connaissent le scriptable object peuvent demander l'instance.

On peut avoir un code “fermé” (via une interface et une implémentation explicite) pour set la référence qui nous intéresse.

De cette manière on peut créer une Reference<T>

Une autre manière d'aborder la notion de dépendance

- En architecture la notion de dépendance est centrale, un composant ne peut pas travailler tout seul dans son coin.
- UML pour rendre visible cette notion
- L'architecture nous permet de connaître et de rendre explicite ces relations
- Viens la problématique de gérer nos dépendances

Singleton

- Je veux obtenir une instance depuis n'importe où YOLO
- Très facile à mettre en place
- **MAIS** brise complètement l'architecture ! Plus d'encapsulation et il faut LIRE le code pour se rendre compte que l'on dépend d'un singleton.

Service Locator - GetComponent

- Permet d'obtenir un composant avec une localité intéressante, GetComponent (attention peu performant)
- FindObject, un peu bourrin et comme le Singleton brise l'architecture (de nos gameobject)
- On obtient facilement un composant

- **MAIS** il faut savoir où se trouve le composant que l'on cherche : GetComponent, GetComponentInChildren, GetComponentInParent. Si l'arborescence change, on doit changer le code.
- Comme le Singleton, on doit lire le code pour comprendre que l'on dépend d'un composant donné.

Champs + Éditeur (aka Injection par champ)

- [SerializeField] sur un champs d'une classe
- Nous permet de rendre lisible la dépendance au sein de la classe, en la lisant en diagonale, on voit direct ses champs et donc les classes qu'elle s'apprête à utiliser, on voit les dépendances également dans l'éditeur.
- On peut set les valeurs au sein d'Unity, si la dépendance change ou est déplacé, il suffit de mettre à jour ce champs, maintenabilité ++
- **MAIS** ne peut pas gérer les références dynamiques (ex : composants instancié au runtime, les références en dehors de prefab etc)
- Si on veut utiliser des interfaces, Unity nous aide pas dans la sérialisation

Méthode d'initialisation (~= injection par constructeur)

- Dans le cas d'un GameObject instancié au runtime, le composant qui gère l'instantiation peut au passage fournir aux composant du prefab tout ou partie de ses dépendances.
 - Ex : on instancie une bullet, on peut ajouter du code pour qu'il ignore certaines hitbox, comme la hitbox du tireur. On peut aussi fournir à la bullet un accès à un manager de son si besoin etc.
- Délègue la responsabilité du maintien des dépendances au composant qui instancie le nouveau GameObject donc il doit pouvoir les accéder (Généralement ce type de composant gère l'ensemble du cycle de vie de ses nouvelles instance, donc Instantiate et Destroy)
- **MAIS** on doit penser à appeler la méthode d'initialisation, on doit aussi connaître les dépendances de nos instances

Conteneur d'injection de dépendance (ex: Zenject)

- Obtenir les dépendances, c'est déjà tout un travail, une responsabilité
- Permet l'accès à des entités importantes
- Extrêmement malléable, dynamique et permet de rendre des tests automatisés facile
- Supporte les interfaces, gère les instances “uniques”, gère des dépendances “contextuelles” etc
- **MAIS** plus complexe à mettre en place, à utiliser si on a bien compris quand l'utiliser, pourquoi et comment

Exemple : Conférence de Niantic :

<https://www.youtube.com/watch?v=8hru629dkRY>

Proxy

Redirection d'un appel vers un tier

Cas de la recherche de composant durant un message de la physique (OnTriggerEnter)

Principe

Quand on récupère un message d'Unity concernant un trigger/collide on est dans le grand flou, on peut avoir collide/trigger avec beaucoup d'éléments dont beaucoup qui nous intéressent pas.

(tips, gérer les layers des GameObject pour rendre plus lisible et recevoir moins d'événements intempestifs)

Typiquement, ma balle touche un gameobject, je veux son composant Health pour infliger des dégâts.

Où est le composant Health ?

- Dans le même GameObject que le collider ? (et dans ce cas GetComponent)
- à un niveau plus haut dans la hiérarchie ? (GetComponentInParent)
- Ailleurs ? (comment récupérer ?)

On crée une forte relation entre la bullet d'une part et Health de l'autre.

L'un doit savoir précisément comment récupérer son copain pour travailler.

Solution potentielle : Proxy

“ T’inquiète, je suis dans le GameObject du Collider et je connais Health, je suis un proxy de Health, donne moi tes appels je les redistribuerais à Health”

On allège le couplage, Health n’est plus obligé d’être dans un GameObject spécifique, le proxy a la responsabilité de connaître le composant (via une injection par champs par exemple)

Une autre utilisation perso

PhysicEvent

Décorrélér les collider/trigger des composants qui les consomment

Design pattern en chaine

Singleton

Bon bah celui là vous le connaissez

Factory Method

Permet de créer différentes entités selon des critères spécifiques :

J'ai 2 équipes qui s'opposent Humain vs Orc, chaque équipe a le même type de soldat mais avec des légères différences par rapport à la race.

On peut donc demander à recevoir un Soldat et recevoir celui qui est bien adapté à ma situation.

Avantage : On reçoit une classe dont on ne connaît pas forcément l'identité exacte.

Qu'importe l'équipe on reçoit une classe Soldat, alors que la

On un peu retrouve ce même principe avec Unity

Vous ne savez pas comment fonctionne l'ajout d'un composant à un `GameObject`.

Vous n'avez aucun autre moyen pour ajouter un composant que de faire appel à la fonction `"AddComponent<T>"`.

Unity seul sait comment gérer l'ajout d'un composant, il a la possibilité d'être au courant de ce nouveau composant, il peut l'enregistrer comme le veut son process etc.

Prototype

Le principe est de cloner une instance qui existe déjà pour la récupérer et l'alterer comme vous le souhaitez.

Relativement classique, arrive souvent pour des ScriptableObjects

Commande

Attention celui la est balaise

C'est ce design pattern qui est mis en place pour pouvoir faire un Ctrl+Z

Chaque opération effectués est symbolisé par un objet.

On a une liste qui correspond donc à notre historique de modification

Chaque objet a une méthode DO et une méthode UNDO, qui vont respectivement faire l'opération et l'annuler.

Commande

Dans le contexte d'un logiciel c'est plus facile à produire (des fois)

Dans un jeu ça nécessite de faire attention à énormément de paramètres :

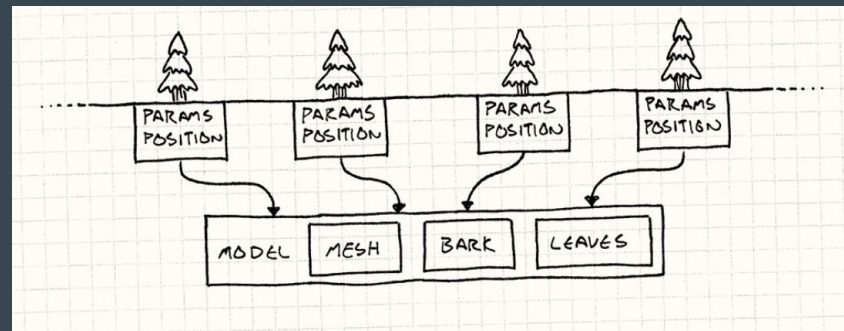
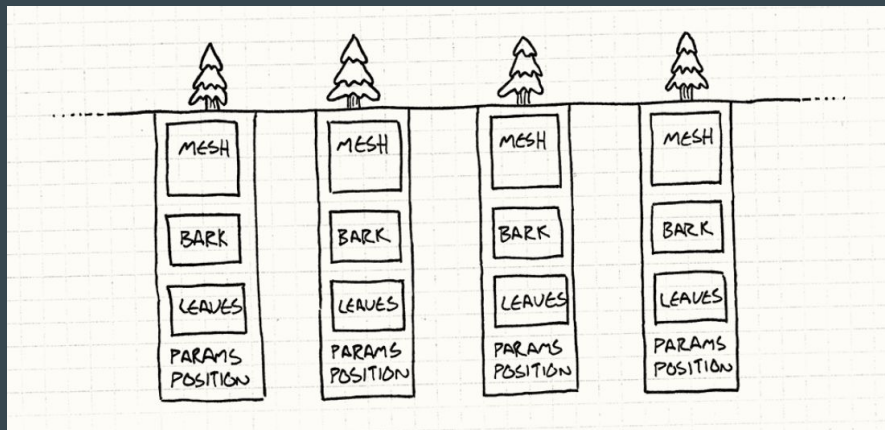
Un saut c'est deux actions : le début du saut, et l'atterrissage. De cette manière si on rollback on sait à partir de quelle position il faut commencer à “sauter dans l'autre sens”.

Il est coutume de sampler la position des entités à interval régulier en plus des actions
(ex: Zelda Tears of the Kingdom)

Flyweight

Pattern d'optimisation, au lieu d'avoir une référence vers sa texture et la charger en mémoire on mutualise et tout les objets concernés ont comme ref la même image.

Optimisation mémoire principalement



source : gameprogrammingpatterns.com

State Machine

Un grand classique pour nous les dev

Permet de gérer le comportement d'une entité par rapport à certaines situations

En StateMachine on peut retrouver par exemple l'Animator d'Unity.

Utile pour de la programmation d'IA (d'autres solutions existent pour de l'IA, behavior tree, GOAP etc)

Dirty flag

L'objet indique lui même s'il necessite une update

=> On évite des update superflue et donc on économise des cycles CPU

Se formalise principalement sous forme d'un simple booléen comme pour la classe Transform.

Egalement lorsque l'on manipule des SO ou d'autre type de fichier, il est de coutume de faire un SetDirty pour indiquer à Unity qu'il doit sauvegarder ou mettre à jour l'information.

Data Locality (c'est plus un principe d'opti qu'un vrai pattern)

La data est très importante que ce soit dans sa quantité qui est gérée que l'endroit où elle est stockée.

Stack = rapide, information directement disponible

Heap = plus lente car il faut sauter de pointeur en pointeur pour accéder à l'info

De cette notion vient la différence entre les struct et les class en C#

Data Locality

De plus, on préfère avoir les informations contiguës en mémoire pour des raisons de communication avec la RAM

Lors d'une demande d'accès à une case d'un tableau par exemple, la RAM nous fournit une plage mémoire dans laquelle il y a la variable demandée.

Si jamais nos infos sont contiguës, lorsque l'on cherche notre 2e element ... il est juste à côté et donc déjà en notre possession via la plage mémoire récupérée !

C'est de là que vient l'idée d'une architecture avec des tableaux de struct

On découpe nos entités sous forme de struct ultra spécialisé (qu'on appelle component). Et on rassemble toutes les struct du même type dans un tableau.

Lors d'une update on a un system spécialisé qui a vocation à parcourir tout ce tableau de component pour mettre tous ces éléments à jour

Entité, Component, System

ECS

DOTS

ECS

Burst Compiler

Jobs

Une architecture avec un code optimisé et une possibilité de paralléliser le travail

Performance ++

En revanche ...

L'approche DOTS est encore en cours de développement. ECS est déjà bien avancé mais une partie des outils d'Unity n'est pas encore ECS compatible (comme l'Animator).

Egalement le fait de split nos entités en micro component fait que l'on perd cette unité que l'on avait avec nos GameObject standards.

2 approches du dev

GameObject :

Stable, éprouvé

Organisation simple et “intuitive”

Adapté à une majorité des jeux

Mais performance -- lorsque l'on a des milliers d'entités à gérer

ECS :

Développement bien avancé (mais)

Organisation séparée en micro composant et système, moins lisible

Adapté à la gestion d'un très grand nombre d'entité

Cas d'utilisation : MMO, jeux avec des milliers d'entités etc.

On peut tout à fait mélanger les deux approches

Garder la structure des GameObjects et optimiser avec ECS les parties qui le méritent.

Mais un jeu 100% ECS sera forcément bien plus performant ... au prix d'une plus grande difficulté de maintenance et d'évolutivité potentiellement

Chaine YT : <https://www.youtube.com/@TurboMakesGames>